

박수민_0629_keypad_calculator

#main.c

```
/*
 * 11.KEYPAD.c
 *
 * Created: 2026-06-29 오후 1:42:36
 * Author : kccistc
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h> // sei, ISR 등
#include <stdio.h>
#include <util/delay.h>
#include "button.h"
#include "queue.h"
#include "cal.h"

void init_timer0();
extern void init_uart0();
extern void init_keypad();
extern void UART0_transmit(uint8_t data);
extern uint8_t keypad_scan(void);
extern void insert_queue(uint8_t value);
extern int queue_empty();
extern uint8_t read_queue();
extern void init_button();
void cal_input(t_cal *c, uint8_t key);
void cal_reset(t_cal *c);

volatile uint32_t keypad_counter=0;
volatile uint32_t msec_count = 0;
volatile uint8_t keypad_flag = 0;

FILE OUTPUT = FDEV_SETUP_STREAM(UART0_transmit, NULL, _FDEV_SETUP_WRITE);

ISR(TIMER0_OVF_vect)
{
```

```

volatile uint8_t keydata = 0;

TCNT0 = 6;          // TCNT0 6~256: 250 펄스 count 하기 위해
msec_count++;

if (++keypad_counter >= 60)
{
    keypad_counter = 0;
    if (keydata = keypad_scan())    //keypad를 체크해서 눌러진 것이 있어야
한다.
    {
        insert_queue(keydata);
        keypad_flag = 1;
    }
};

}

int main(void)
{
    t_cal cal;
    cal_reset(&cal);

    init_timer0();
    init_uart0();
    init_keypad();
    init_button();

    stdout = &OUTPUT;          // printf가 동작할 수 있도록 stdout을 설정
    sei();

    while (1)
    {
        if (keypad_flag)
        {
            keypad_flag = 0;

            uint8_t key = keypad_scan();    // 여기서 처리
            if (key) insert_queue(key);

            if (get_button(BUTTON0, BUTTON0PIN)) insert_queue(' ');

```

```

        if (get_button(BUTTON1, BUTTON1PIN)) insert_queue('');
        if (get_button(BUTTON2, BUTTON2PIN)) insert_queue('C');
    }

    if (!queue_empty())
    {
        //key_value = read_queue();
        uint8_t key = read_queue();
        cal_input(&cal, key);
        //printf("key_value: %c\n", key_value);
    }
}

void init_timer0()
{
    // TCNT0 = TimerCount0
    // TCNT0 6 ~ 256 : 250개 펄스를 count 하기 위해 6부터 시작
    TCNT0 = 6;

    TCCR0 &= ~( 1<< CS02 | 1 << CS01 | 1 << CS00); // 0분주 reset 먼저
    TCCR0 |= 1 << CS02 | 0 << CS01 | 0 << CS00;      // 64분주
    TIMSK |= 1 << TOIE0;    // TIMER0 Overflow INT
}

```

```

#keypad.c
/*
 * keypad.c
 *
 * Created: 2026-06-29 오후 2:07:57
 * Author: kccistc
 */

#include "keypad.h"

void init_keypad(void);
uint8_t keypad_scan(void);
uint8_t get_keypad(int row, int col);

void init_keypad()
{
    KEYPAD_DDR = 0x0f;    // row: 입력 col: 출력
    KEYPAD_PORT = 0xff; // 전체 HIGH (풀업 + COL 비활성)
}

uint8_t keypad_scan(void)
{
    uint8_t data = 0;

    for (int col = 0; col < 4; col++)
    {

        for (int row = 0; row < 4; row++)
        {
            data = get_keypad(row, col);
            if (data) return data;
        }
    }
    return 0;
}

uint8_t get_keypad(int row, int col)
{
    uint8_t keypad_char[4][4] = {
        {'/', '3', '2', '1'},
        {'*', '6', '5', '4'},
        {'-', '9', '8', '7'},
    };
}

```

```

        {'+', '=', '0', ' ' }
};

static int8_t prev_state[4][4] =
{
    { 1, 1, 1, 1 },           // 초기버튼은 눌러지지 않은 상태로 한다.
    { 1, 1, 1, 1 },
    { 1, 1, 1, 1 },
    { 1, 1, 1, 1 }
};

int8_t current_state = 1;
KEYPAD_PORT = 0xff;
KEYPAD_PORT &= ~(1 << col); // 해당 col에 전류를 흘린다.

for (int delay = 0; delay < 20; delay++); // keypad 체크를 위한 delay(0.625us * 20
= 1.25us)

current_state = ( KEYPAD_PIN & (1 << (row+4)) ) >> (row+4);
// 예: row0: pa4에 연결
// 76543210
// 11101111 KEYPAD_PIN & (1 << (row+4))
// 00001110

if (current_state == 0 && prev_state[row][col] == 1) // 처음 눌러진 상태
{
    prev_state[row][col] = 0;
    return 0;
}
else if (current_state == 1 && prev_state[row][col] == 0)
{
    // 이전에 버튼이 눌러지고 현재는 버튼을 떼 상태이면 버튼을 1번 눌렀다 떼
    것으로 인정
    prev_state[row][col] = 1; // prev_state를 초기화
    // printf("key: %c\n", keypad_char[row][col]);
    return keypad_char[row][col];
}

return 0;
}

```

```
#keypad.h
/*
 * keypad.h
 *
 * Created: 2026-06-29 오후 2:08:08
 * Author: kccistc
 */

#ifndef KEYPAD_H_
#define KEYPAD_H_
#define F_CPU 16000000UL // 16MHz
#include <avr/io.h> // PORTA PORTB PORTD... IO관련 reg가 들어 있다.
#include <util/delay.h> // _delay_ms _delay_us 등
#include <avr/interrupt.h>
#include <string.h>
#include <stdio.h>

#define KEYPAD_DDR DDRA // 방향설정
#define KEYPAD_PIN PINA
#define KEYPAD_PORT PORTA

#endif /* KEYPAD_H_ */
```

```

#cal.c
/*
 * cal.c
 *
 * Created: 2026-06-29 오후 4:08:20
 * Author: kccistc
 */
#include "cal.h"
#include "keypad.h"

void cal_main(void);
void cal_input(t_cal *c, uint8_t key);
void cal_reset(t_cal *c);
int32_t cal_evaluate(char *expr); // Shunting-yard 계산

extern void init_keypad();
extern uint8_t keypad_scan(void);

static uint8_t precedence(char op)
{
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

// 계산 수행
static int32_t apply_op(int32_t a, int32_t b, char op)
{
    switch (op)
    {
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;
        case '/':
            if (b == 0) { printf("ERR: div by zero\n"); return 0; }
            return a / b;
    }
    return 0;
}

int32_t cal_evaluate(char *expr)
{

```

```

int32_t num_stack[STACK_SIZE]; // 숫자 스택
char    op_stack[STACK_SIZE];  // 연산자 스택
int8_t  num_top = -1;
int8_t  op_top  = -1;

int8_t i = 0;

while (expr[i] != '\0')
{
    // 1. 숫자면 파싱해서 num_stack에 push
    if (expr[i] >= '0' && expr[i] <= '9')
    {
        int32_t num = 0;
        while (expr[i] >= '0' && expr[i] <= '9')
        {
            num = num * 10 + (expr[i] - '0');
            i++;
        }
        num_stack[++num_top] = num;
        continue;
    }

    // 2. '(' 이면 op_stack에 push
    if (expr[i] == '(')
    {
        op_stack[++op_top] = expr[i];
        i++;
        continue;
    }

    // 3. ')' 이면 '(' 나올 때까지 계산
    if (expr[i] == ')')
    {
        while (op_top >= 0 && op_stack[op_top] != '(')
        {
            int32_t b = num_stack[num_top--];
            int32_t a = num_stack[num_top--];
            char    op = op_stack[op_top--];
            num_stack[++num_top] = apply_op(a, b, op);
        }
        op_top--; // '(' 제거
        i++;
        continue;
    }
}

```



```

    }

    // 4. 연산자면 우선순위 비교 후 처리
    if (expr[i]=='+' || expr[i]=='-' || expr[i]=='*' || expr[i]=='/')
    {
        while (op_top >= 0 && op_stack[op_top] != '(' &&
precedence(op_stack[op_top]) >= precedence(expr[i]))
        {
            int32_t b = num_stack[num_top--];
            int32_t a = num_stack[num_top--];
            char op = op_stack[op_top--];
            num_stack[++num_top] = apply_op(a, b, op);
        }
        op_stack[++op_top] = expr[i];
        i++;
        continue;
    }

    i++; // 모르는 문자 스킵
}

// 5. 남은 연산자 전부 처리
while (op_top >= 0)
{
    int32_t b = num_stack[num_top--];
    int32_t a = num_stack[num_top--];
    char op = op_stack[op_top--];
    num_stack[++num_top] = apply_op(a, b, op);
}

return num_stack[num_top];
}

void cal_reset(t_cal *c)
{
    memset(c->buf, 0, BUF_SIZE);
    c->idx = 0;
    printf("CLEAR\n");
}

void cal_input(t_cal *c, uint8_t key)
{
    // Clear

```

```

if (key == 'C')
{
    cal_reset(c);
    return;
}

// '=' 누르면 계산
if (key == '=')
{
    if (c->idx == 0)
    {
        printf("error: empty\n");
        return;
    }
    printf("%s=", c->buf);
    int32_t result = cal_evaluate(c->buf);
    printf("%ld\n", result);
    cal_reset(c);
    return;
}

// 버퍼에 추가
if (c->idx < BUF_SIZE - 1)
{
    c->buf[c->idx++] = key;
    c->buf[c->idx] = '\0';
    printf("%s\n", c->buf); // 현재 입력 상태 출력
}
else
{
    printf("error: buf full\n");
}
}

```

```
#cal.h
/*
 * cal.h
 *
 * Created: 2026-06-29 오후 4:08:43
 * Author: kccistc
 */

#ifndef CAL_H_
#define CAL_H_

#include <avr/io.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BUF_SIZE 32
#define STACK_SIZE 16

typedef struct _cal
{
    char    buf[BUF_SIZE];    // 입력 버퍼 ex) "10+20*3"
    uint8_t idx;
} t_cal;

#endif /* CAL_H_ */
```

키패드 계산 결과

```
1
10
10+
10+2
10+23
10+23=33
CLEAR
1
10
10-
10-2
10-23
10-23=-13
CLEAR
1
10
100
100/
100/1
100/10
100/10=10
CLEAR
1
10
10+
10+2
10+20
10+20*
10+20*1
10+20*10
10+20*10/
10+20*10/1
10+20*10/15
10+20*10/15=23
CLEAR
```